

OverTheWire — Bandit: Guía completa

Niveles 0–19 con explicación de cada comando

Juan David Rangel

Índice

1. Nivel 0 – Primer contacto	2
2. Nivel 1 – El archivo llamado -	2
3. Nivel 2 – Archivo con espacios en el nombre	3
4. Nivel 3 – Archivo oculto	4
5. Nivel 4 – El archivo legible por humanos	4
6. Nivel 5 – Buscar por propiedades	5
7. Nivel 6 – Buscar en todo el sistema	6
8. Nivel 7 – grep	7
9. Nivel 8 – La línea única	7
10. Nivel 9 – strings	8
11. Nivel 10 – base64	8
12. Nivel 11 – ROT13	9
13. Nivel 12 – Cadena de compresiones	9
14. Nivel 13 – Clave privada SSH	11
15. Nivel 14 – Netcat	12
16. Nivel 15 – Servicio SSL/TLS	12
17. Nivel 16 – Escaneo de puertos + TLS	13
18. Nivel 17 – diff	14
19. Nivel 18 – Sesión que se cierra sola	15
20. Nivel 19 – Binario setuid	15

Guía paso a paso de los primeros 20 niveles de Bandit con capturas de terminal renderizadas, explicación detallada de cada comando y la contraseña para el siguiente nivel.

Cómo conectarse

```
ssh -p 2220 bandit<N>@bandit.labs.overthewire.org
```

Nota: Nota sobre el puerto: SSH usa por defecto el puerto 22, pero Bandit corre en el puerto **2220** para no interferir con el servicio normal del servidor. Por eso siempre hay que indicarlo con `-p 2220`. La contraseña del nivel 0 es simplemente **bandit0**.

1. Nivel 0 – Primer contacto

Objetivo: conectarse por SSH y leer el archivo `readme` que está en el directorio personal.
Comando:

```
cat readme
```



```
bandit0@bandit:~$ cat readme
Congratulations on your first steps into the bandit game!!
Please make sure you have read the rules at https://overthewire.org/rules/
If you are following a course, workshop, walkthrough or other educational activity,
please inform the instructor about the rules as well and encourage them to
contribute to the OverTheWire community so we can keep these games free!

The password you are looking for is: 6y2kwnwK6grgvwvpvLaa2T1cpFEK0hNR
```

Explicación:

- `cat` (del inglés *concatenate*) imprime el contenido de un archivo en la salida estándar. Es la herramienta más básica para leer archivos de texto en Linux.
- Al iniciar sesión, el shell te deja en el directorio personal (`/home/bandit0`), que contiene un único archivo llamado `readme` con la contraseña del nivel 1.
- Concepto importante: si el archivo no existe, `cat` muestra un error de tipo *No such file or directory*. Leer bien los errores es parte del oficio.

Contraseña para bandit1: 6y2kwnwK6grgvwvpvLaa2T1cpFEK0hNR

2. Nivel 1 – El archivo llamado -

Objetivo: leer un archivo cuyo nombre es un guion solo (`-`). **Comando:**

```
cat ./-
```

```
bandit1@bandit:~$ cat ./-
PK8fYLZg2hnHSz83p1BL1iEPKdD3QToB
```

Explicación:

- El guion (-) es un carácter reservado en muchas herramientas de Unix: significa *entrada estándar*. Si ejecutás `cat -`, el programa espera a que escribas por teclado en lugar de leer un archivo.
- La solución es desambiguar con la ruta explícita: `./-` indica ".el archivo llamado - dentro del directorio actual". El prefijo `./` elimina cualquier interpretación especial.
- Lección: los nombres de archivo pueden contener cualquier carácter salvo / y el nulo; los caracteres "especiales" se neutralizan con rutas explícitas o comillas (ver nivel 2).

Contraseña para bandit2: PK8fYLZg2hnHSz83p1BL1iEPKdD3QToB

3. Nivel 2 – Archivo con espacios en el nombre

Objetivo: leer un archivo cuyo nombre contiene espacios. **Comandos:**

```
ls -la
cat -- '--spaces in this filename--'
```

```
bandit2@bandit:~$ ls -la
bandit2@bandit:~$ cat -- '--spaces in this filename--'
total 24
-rw-r----- 1 bandit3 bandit2  33 Jun 24 14:59 --spaces in this filename--
drwxr-xr-x  2 root    root    4096 Jun 24 14:59 .
drwxr-xr-x 150 root    root    4096 Jun 24 15:02 ..
-rw-r--r--  1 root    root     220 Feb 13 12:16 .bash_logout
-rw-r--r--  1 root    root    3851 Jun 24 14:50 .bashrc
-rw-r--r--  1 root    root     807 Feb 13 12:16 .profile
7ZZ2LFrykP2zEyvB14m3clL7tGYJPME
```

Explicación:

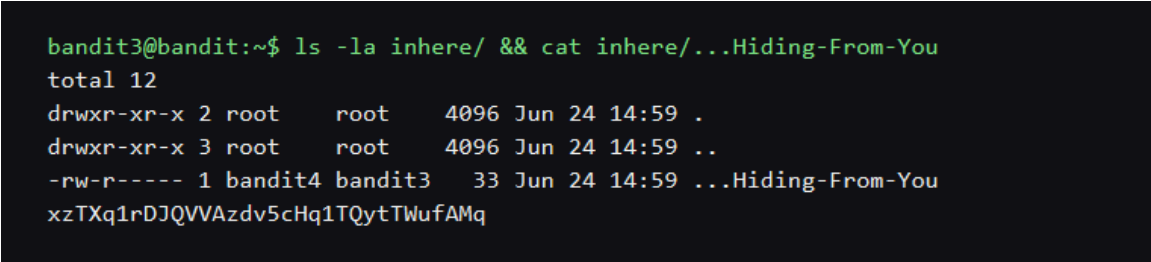
- `ls -la` lista todos los archivos (incluso ocultos, por la `-a`) con detalles (por la `-l`). Aquí se ve el nombre real: `--spaces in this filename--`, que tiene guiones **al inicio y al final**.
- Las **comillas simples** agrupan todo lo que está entre ellas en un único argumento, de modo que los espacios no se interpreten como separadores. Sin comillas, `cat` recibiría cinco argumentos sueltos (`spaces`, `in`, `this`, `filename`).
- El doble guion (`--`) es la convención universal de GNU para "fin de opciones": le dice a `cat` que todo lo que sigue es un archivo y no una bandera. Sin él, un nombre que empieza con `-` se interpretaría como opción (prueba de ello: `cat` falla con *unexpected argument*).
- `ls -la` es el primer reflejo de cualquier persona que trabaja en un sistema Linux: ver qué hay, con qué permisos y qué oculto.

Contraseña para bandit3: 7ZZ2LFrykP2zEyvB14m3c1cL7tGYJPME

4. Nivel 3 – Archivo oculto

Objetivo: encontrar y leer el archivo oculto dentro de `inhere/`. **Comandos:**

```
ls -la inhere/  
cat inhere/...Hiding-From-You
```



```
bandit3@bandit:~$ ls -la inhere/ && cat inhere/...Hiding-From-You  
total 12  
drwxr-xr-x 2 root    root    4096 Jun 24 14:59 .  
drwxr-xr-x 3 root    root    4096 Jun 24 14:59 ..  
-rw-r----- 1 bandit4 bandit3   33 Jun 24 14:59 ...Hiding-From-You  
xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq
```

Explicación:

- En Linux, un archivo es **oculto** cuando su nombre empieza con un punto (`.`). `ls` a secas no lo muestra; `ls -a` (o `-la`) sí.
- El truco de este nivel es doble: además de estar oculto, el nombre empieza con **tres puntos** (`...Hiding-From-You`), lo que lo hace fácil de pasar por alto incluso mirando el listado.
- `ls -la inhere/` muestra los permisos y el propietario: el archivo pertenece al usuario `bandit4`, que es justamente el siguiente nivel – pista clásica de Bandit.
- Lección: cuando un desafío dice "hay un archivo oculto", lo primero es `ls -la`; y si el nombre empieza con puntos, mirá con atención.

Contraseña para bandit4: xzTXq1rDJQVVAzdv5cHq1TQytTWufAMq

5. Nivel 4 – El archivo legible por humanos

Objetivo: entre diez archivos de `inhere/`, encontrar el único que es texto plano legible. **Comandos:**

```
file inhere/*  
cat inhere/-file07
```

```
bandit4@bandit:~$ file inhere/* && cat inhere/-file07
inhere/-file00: data
inhere/-file01: data
inhere/-file02: OpenPGP Secret Key
inhere/-file03: data
inhere/-file04: data
inhere/-file05: data
inhere/-file06: Non-ISO extended-ASCII text, with NEL line terminators
inhere/-file07: ASCII text
inhere/-file08: data
inhere/-file09: data
6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG
```

Explicación:

- `file` identifica el **tipo real** de un archivo inspeccionando su contenido (los llamados *magic bytes* de los primeros bytes), no su extensión ni su nombre. Es la herramienta correcta cuando el contenido es desconocido.
- El comodín `*` se expande a todos los archivos del directorio, así que `inhere/*` analiza los diez de una vez.
- El resultado: la mayoría son `data` (binarios sin tipo reconocible), hay un par de casos raros (`OpenPGP Secret Key`, texto con caracteres extendidos) y solo `-file07` es `ASCII text` – texto plano puro. Ese es el que contiene la contraseña.
- `cat inhere/-file07` la muestra. Nota: el archivo también empieza con `-`, pero `inhere/` ya desambigua la ruta.

Contraseña para bandit5: 6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG

6. Nivel 5 – Buscar por propiedades

Objetivo: localizar un archivo de exactamente 1033 bytes, que **no** sea ejecutable, en algún lugar del árbol `inhere/`. **Comandos:**

```
find inhere/ -type f -size 1033c ! -executable
cat inhere/maybehere07/.file2
```

```
bandit5@bandit:~$ find inhere/ -type f -size 1033c ! -executable && cat inhere/maybehere07/.file2
inhere/maybehere07/.file2
pXa26xhMWaC2SvDotA4r9EgZkul0eSBW
```

Explicación:

- `find` recorre directorios de forma recursiva aplicando filtros. Es la herramienta definitiva para *encontrar algo cuando no sé dónde está*.
- Los filtros usados:
 - `-type f` → solo archivos regulares (excluye directorios, enlaces, etc.).

- `-size 1033c` → tamaño exacto de 1033 bytes. La `c` es la unidad: *bytes*. Sin unidad, `find` interpreta bloques de 512 bytes.
- `! -executable` → negación: NO debe tener bit de ejecución.
- El único resultado es un archivo **oculto** (`maybehere07/.file2`) en un subdirectorío profundo – otro recordatorio de que `ls -la` es tu amigo.
- Lección: `find` con `-size`, `-user`, `-group`, `-type` y negaciones (!) resuelve la mayoría de búsquedas "por características".

Contraseña para bandit6: pXa26xhMWaC2SvDotA4r9EgZkul0eSBW

7. Nivel 6 – Buscar en todo el sistema

Objetivo: encontrar un archivo de 33 bytes propiedad del usuario `bandit7` y del grupo `bandit6`, en cualquier parte del sistema. **Comando:**

```
F=$(find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null)
echo "RUTA: $F"
cat $F
```

```
bandit6@bandit:~$ F=$(find / -type f -user bandit7 -group bandit6 -size 33c 2>/dev/null); echo "RUTA: $F"; cat $F
RUTA: /var/lib/dpkg/info/bandit7.password
Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3
```

Explicación:

- La búsqueda ahora arranca en `/` (raíz del sistema de archivos completo), no en el directorio actual.
- Filtros nuevos: `-user bandit7` (propietario) y `-group bandit6` (grupo). Combinados con `-type f` y `-size 33c`, queda una sola coincidencia.
- `2>/dev/null` redirige los **errores estándar** (`stderr`) a la papelera: al recorrer todo el sistema, `find` intenta entrar en directorios sin permiso y escupe muchísimos *Permission denied*. Silenciarlos permite ver solo el resultado útil. Este patrón (comando `2>/dev/null`) es ubicuo en administración de sistemas.
- La variable `F` guarda la ruta encontrada (`$(...)` es *sustitución de comando*: ejecuta y captura la salida) para poder leerla después con `cat`.
- El archivo vive en `/var/lib/dpkg/info/bandit7.password` – los niveles de Bandit suelen esconder archivos entre los datos de paquetes del sistema.

Contraseña para bandit7: Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3

8. Nivel 7 – grep

Objetivo: encontrar, dentro de `data.txt`, la línea que contiene la palabra `millionth`. **Comando:**

```
grep millionth data.txt
```

```
bandit7@bandit:~$ grep millionth data.txt
millionth      VR1ljMayciFxbnUokuQmJFw6QC9VKtub
```

Explicación:

- `grep` busca patrones dentro de archivos e imprime las líneas que coinciden. Es probablemente el comando más usado de Linux después de `ls`.
- El archivo tiene miles de líneas con el formato `palabra<TAB>password`. Pedir `grep millionth` devuelve exactamente la línea que nos interesa.
- La salida muestra el separador tabulador: `millionth` seguido de la contraseña en el mismo renglón.
- Sintaxis básica: `grep PATRÓN ARCHIVO`. Los patrones pueden ser expresiones regulares (ahí se vuelve realmente poderoso).

Contraseña para bandit8: `VR1ljMayciFxbnUokuQmJFw6QC9VKtub`

9. Nivel 8 – La línea única

Objetivo: `data.txt` contiene muchas líneas repetidas; encontrar la única que aparece una sola vez. **Comando:**

```
sort data.txt | uniq -u
```

```
bandit8@bandit:~$ sort data.txt | uniq -u
EjmOSvuAu7sGAHqHVcBDPirRe9T03kx1
```

Explicación:

- La tubería (`|`) conecta la salida de un comando con la entrada del siguiente: `sort` ordena todas las líneas y su salida alimenta a `uniq`.
- `uniq` solo detecta líneas **adyacentes** duplicadas; por eso primero hay que ordenar: las repeticiones quedan juntas.
- `uniq -u` (de *unique*) imprime únicamente las líneas que aparecen una sola vez. Como la contraseña es la única línea no repetida, es lo único que sale.
- Concepto central de Unix: **componer herramientas pequeñas** con tuberías en lugar de escribir programas grandes. `sort | uniq` es un clásico.

Contraseña para bandit9: EjmOSvuAu7sGAHqHVcBDPirRe9T03kx1

10. Nivel 9 – strings

Objetivo: encontrar la contraseña dentro de un archivo binario. **Comando:**

```
strings data.txt | grep '=='
```

```
bandit9@bandit:~$ strings data.txt | grep '=='
===== the
[==p+
===== password
Y===== is
===== B0s2khmbT9u0geKu0oVGW3JZKhndE3BG
```

Explicación:

- `data.txt` es un archivo binario: `cat` lo mostraría como basura ilegible. `strings` extrae las secuencias de caracteres imprimibles que contiene (todo archivo binario arrastra fragmentos de texto).
- El filtro `grep '=='` conserva solo las líneas que parecen marcas de contraseña. La salida muestra un patrón armado: `===== the`, `===== password`, `===== is`, y por fin `===== B0s2k...` – la contraseña.
- Lección de seguridad: los datos `.ocultos.en` binarios se recuperan con `strings`; por eso los programas serios no guardan secretos en texto plano ni en el binario.

Contraseña para bandit10: B0s2khmbT9u0geKu0oVGW3JZKhndE3BG

11. Nivel 10 – base64

Objetivo: decodificar el contenido del archivo. **Comando:**

```
base64 -d data.txt
```

```
bandit10@bandit:~$ base64 -d data.txt
The password is pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro
```

Explicación:

- Base64 es una codificación que representa datos binarios con un alfabeto de 64 caracteres imprimibles (letras, dígitos, + y /). Se usa mucho en la web (adjuntos, tokens JWT, etc.).
- `base64` sin opciones **codifica**; `base64 -d` (de *decode*) **decodifica**.

- El archivo contiene una frase completa codificada: al decodificarla aparece `The password is pYf0Y....`
- Nota: base64 **no es cifrado** – es codificación, cualquiera puede revertirla sin clave. Este nivel enseña a reconocerla: bloques de letras/dígitos que terminan en `=`.

Contraseña para bandit11: `pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro`

12. Nivel 11 – ROT13

Objetivo: decodificar un cifrado César de 13 posiciones. **Comando:**

```
tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt
```

```
bandit11@bandit:~$ tr 'A-Za-z' 'N-ZA-Mn-za-m' < data.txt
The password is GR0ozWP08QyN0mGrjUkID0WCYkZiQxrN
```

Explicación:

- ROT13 es un cifrado César que desplaza cada letra 13 posiciones en el alfabeto ($A \rightarrow N$, $B \rightarrow O$, ... $M \rightarrow Z$, $N \rightarrow A$, ...). Es el cifrado "de juguete" clásico de Usenet.
- `tr` (de *translate*) reemplaza caracteres del primer conjunto por los del segundo, uno a uno:
 - `A-Za-z` $\rightarrow$ todas las letras mayúsculas y minúsculas.
 - `N-ZA-Mn-za-m` $\rightarrow$ el alfabeto desplazado 13 ($N-Z + A-M$ para mayúsculas, igual para minúsculas).
- La redirección de entrada (`<`) hace que `tr` lea de `data.txt` en lugar del teclado.
- Propiedad elegante: ROT13 aplicado dos veces vuelve al texto original, así que **decodificar** = **codificar** con el mismo comando.
- Lección: distinguir cifrado real (con clave) de ofuscaciones triviales que cualquier `tr` revierte.

Contraseña para bandit12: `GR0ozWP08QyN0mGrjUkID0WCYkZiQxrN`

13. Nivel 12 – Cadena de compresiones

Objetivo: recuperar la contraseña desde un volcado hexadecimal que, invertido, revela un archivo comprimido repetidamente con distintas herramientas. **Comandos:**

```
xxd -r data.txt > f0
file f0
gzip -dc f0 > f1
bzip2 -dc f1 > f2
gzip -dc f2 > f3
tar xf f3
```

```
tar xf data5.bin
bzip2 -dc data6.bin > f4
tar xf f4
gzip -dc data8.bin > f5
cat f5
```

```
bandit12@bandit:~$ sh /tmp/b12doc/b12doc.sh
f0: gzip compressed data, was "data2.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modulo 2^32 580
f1: bzip2 compressed data, block size = 900k
f2: gzip compressed data, was "data4.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modulo 2^32 20480
f3: POSIX tar archive (GNU)
data.txt
data5.bin
f0
f1
f2
f3
data5.bin: POSIX tar archive (GNU)
data.txt
data5.bin
data6.bin
f0
f1
f2
f3
data6.bin: bzip2 compressed data, block size = 900k
f4: POSIX tar archive (GNU)
data.txt
data5.bin
data6.bin
data8.bin
f0
f1
f2
f3
f4
== data5.bin: POSIX tar archive (GNU)
== data6.bin: bzip2 compressed data, block size = 900k
== data8.bin: gzip compressed data, was "data9.bin", last modified: Wed Jun 24 14:58:46 2026, max compression, from Unix, original size modulo 2^32 49
f5: ASCII text
The password is qQYQiHOBPR8zR61qxYqX45quvihF2uzk
```

Explicación:

- `xxd -r` (de *reverse*) convierte el volcado hexadecimal de vuelta a binario. La salida se guarda en `f0` con la redirección `>`.
- A partir de ahí comienza un ciclo: `file` dice qué tipo es, y se descomprime con la herramienta correspondiente:
 - `gzip -dc`: `-d` descomprime, `-c` escribe en la salida estándar (en vez de crear un archivo).
 - `bzip2 -dc`: lo mismo para el formato bzip2 (más compresión, más lento).
 - `tar xf`: `tar` empaqueta archivos; `x` extrae, `f` indica el archivo. Al extraer aparecen `data5.bin`, `data6.bin`, `data8.bin`...
- La cadena completa es: `hex` → `gzip` → `bzip2` → `gzip` → `tar` → `tar` → `bzip2` → `tar` → `gzip` → **texto ASCII**.
- Lección: `file` antes de descomprimir evita adivinar. En el mundo real, recibir datos de fuentes desconocidas sin saber su formato es moneda corriente, y esta secuencia de detección es la práctica correcta.

Contraseña para bandit13: qQYQiHOBPR8zR61qxYqX45quvihF2uzk

14. Nivel 13 – Clave privada SSH

Objetivo: usar la clave privada `sshkey.private` para iniciar sesión como `bandit14` y leer su contraseña. **Comandos:**

```
ls -la sshkey.private
ssh -i sshkey.private -p 2220 bandit14@bandit.labs.overthewire.org
cat /etc/bandit_pass/bandit14
```

```
bandit13@bandit:~$ ls -la sshkey.private
-rw-r----- 1 bandit14 bandit13 2602 Jun 24 14:58 sshkey.private

C:\Users\RANGE> ssh -i bandit13key -p 2220 bandit14@bandit.labs.overthewire.org

      _ _ _ _ _
      | | _ _ _ _ _ | ( ) |
      | ' \ / _ ' \ / _ ' \ / _ ' \
      | | | | | | | | | | | |
      | _ _ / \ _ _ / \ _ _ / \ _ _

This is an OverTheWire game server.
More information on http://www.overthewire.org/wargames

backend: gibson-1
aaWecNkG4FhxJQxz07uiwzVP6bJiYS65
```

Explicación:

- El directorio personal contiene una **clave privada SSH**: `sshkey.private`. El listado muestra que pertenece al usuario `bandit14`, con permisos `-rw-r-----` (legible por el grupo `bandit13`).
- `ssh -i clave` autentica con esa clave en lugar de pedir contraseña. Las claves SSH reemplazan el secreto compartido por un par criptográfico: la clave pública vive en el servidor y la privada (que jamás debe compartirse) en tu máquina.
- **Detalle importante:** OpenSSH rechaza claves con permisos demasiado abiertos (por seguridad), así que suele ser necesario `chmod 600 sshkey.private`.
- En este caso concreto, conectar a `localhost` desde dentro del servidor de juego está bloqueado; la clave se usa desde la máquina local contra `bandit.labs.overthewire.org`.
- `cat /etc/bandit_pass/bandit14` lee el archivo donde el servidor guarda la contraseña del nivel – nótese que todos los passwords de Bandit viven en `/etc/bandit_pass/`.

Contraseña para bandit14: aaWecNkG4FhxJQxz07uiwzVP6bJiYS65

15. Nivel 14 – Netcat

Objetivo: enviar la contraseña actual a un servicio que escucha en el puerto 30000 de localhost. **Comando:**

```
echo 'aaWecNkG4FhxJQxz07uiwzVP6bJiYS65' | nc localhost 30000
```

```
bandit14@bandit:~$ echo 'aaWecNkG4FhxJQxz07uiwzVP6bJiYS65' | nc localhost 30000
Correct!
pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
```

Explicación:

- nc (netcat) es la "navaja suiza" de las redes: abre una **conexión TCP cruda** con un host y un puerto, sin protocolo de aplicación. Útil para probar servicios y para este tipo de desafíos.
- echo '...' imprime la contraseña y la tubería la envía por la conexión. El servicio, al recibir el texto correcto, responde **Correct!** y la contraseña del siguiente nivel.
- Concepto: un servicio de red no es más que un programa que escucha en un puerto y procesa bytes. Con nc podés hablarle .a mano.a cualquier puerto, igual que el telnet clásico.
- Si el texto no es el esperado, el servicio responde **Wrong!** – pista para revisar qué se está enviando.

Contraseña para bandit15: pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7

16. Nivel 15 – Servicio SSL/TLS

Objetivo: enviar la contraseña al servicio protegido por TLS del puerto 30001. **Comando:**

```
echo 'pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7' | openssl s_client -connect localhost:30001 -quiet
```

```
bandit15@bandit:~$ echo 'pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7' | openssl s_client -connect localhost:30001 -quiet 2>/dev/null
Correct!
kS0Hf0u5HiXFwKMKFqXvPd0TNGGa0X8V
```

Explicación:

- El puerto 30001 **no** es texto plano: habla TLS (el cifrado que protege HTTPS). nc no serviría porque el servicio espera primero el handshake criptográfico.
- openssl s_client es el cliente de prueba TLS de OpenSSL: establece el handshake y luego deja pasar bytes en claro por el canal cifrado.
- -connect localhost:30001 indica destino; -quiet suprime todo el ruido del certificado y deja solo la conversación útil.

- Al igual que el nivel 14, **echo** + tubería envía la contraseña y el servicio responde **Correct!** + la siguiente contraseña.
- Lección: cuando un servicio "no responde con nc, probar si está cifrado con openssl s_client es el siguiente paso natural.

Contraseña para bandit16: kS0Hf0u5HiXFwKMKFqXvPd0TNGGa0X8V

17. Nivel 16 – Escaneo de puertos + TLS

Objetivo: encontrar, entre los puertos 31000 – 32000, el servicio SSL que al recibir la contraseña devuelve la clave privada del siguiente nivel. **Comandos:**

```
nmap -p 31000-32000 localhost
echo '<password>' | openssl s_client -connect localhost:<PUERTO> -quiet
```

```
bandit16@bandit:~$ sh /tmp/b16doc.sh
Starting Nmap 7.98 ( https://nmap.org ) at 2026-08-02 20:42 +0000
Nmap scan report for localhost (127.0.0.1)
Host is up (0.00013s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 996 closed tcp ports (conn-refused)
PORT      STATE SERVICE
31046/tcp  open  unknown
31518/tcp  open  unknown
31691/tcp  open  unknown
31790/tcp  open  unknown
31960/tcp  open  unknown

Nmap done: 1 IP address (1 host up) scanned in 0.06 seconds
=====
--- PORT 31046

--- PORT 31518
kS0Hf0u5HiXFwKMKFqXvPd0TNGGa0X8V

--- PORT 31691

--- PORT 31790
Correct!
-----BEGIN OPENSSH PRIVATE KEY-----
b3BlbnNzaC1rZXktdjEAAAABG5vbmUAAAAEbm9uZQAAAAAAAAABAAABlwAAAAdzc2gtcn
NhAAAAAwEAAQAAAYEAvdSaw8j1FQ2DjtbQPGiEVtqEG5kt3g71uDlixg42vRN2MvWRVnGQ
t4k9T9tDWaisnn+6I4RCkhEzw231WA6Kvc0Sd0+6/6Cp1Egp4o4l+xf5gPNo7A20qjqN67
Hhy6I71GBjyUBnp6vEtkI3WZmZtuxpCMPyHSy7m56lipJFddKEOUCX21hNWWy2SAZQFBub

--- PORT 31960
```

Explicación:

- `nmap` es el escáner de puertos por excelencia. `-p 31000-32000` limita el rango y `localhost` el objetivo. El resultado: cinco puertos abiertos (31046, 31518, 31691, 31790, 31960) y el resto cerrados.
- Tener un puerto **abierto** no dice nada sobre el **protocolo**: hay que probar cada uno. Los cinco se recorren con `openssl s_client` enviando la contraseña.
- Resultado del recorrido: la mayoría no responde o solo repite el eco; el puerto 31790 responde **Correct!** y devuelve una **clave privada OpenSSH** completa (-----BEGIN OPENSSH PRIVATE KEY-----).
- Con esa clave se aplica la técnica del nivel 13 para entrar como `bandit17`.
- Lección: "puerto abierto" ≠ "servicio útil". Escanear y luego identificar el protocolo (con `file` para archivos, `s_client/nc` para servicios) es el método.

Contraseña para **bandit17**: `pWXMAZoxGC8JmDMfmT5MGEsobMM3vnj2`

18. Nivel 17 – diff

Objetivo: encontrar la única línea que cambió entre `passwords.old` y `passwords.new`. **Comando:**

```
diff passwords.old passwords.new
```

```
bandit17@bandit:~$ diff passwords.old passwords.new
42c42
< q0g5pV0jPx9x9VccyYBADiT4xxyoUB8D
---
> 0QxXZjELndr90zuh0TDYBEomI0SZITXI
```

Explicación:

- `diff` compara dos archivos línea por línea y muestra las diferencias en un formato estándar.
- La notación `42c42` significa: en la línea 42 de ambos archivos hubo un **cambio** (la `c`). Otras notaciones: `a` (añadido) y `d` (borrado).
- Las líneas siguientes: `<` es el contenido **viejo** (de `passwords.old`) y `>` el contenido **nuevo** (de `passwords.new`). Entre ambos, una línea `---` separa los dos lados.
- La contraseña es la línea que existe solo en el archivo nuevo: `0QxXZjELndr90zuh0TDYBEomI0SZITXI`.
- `diff` es la base de los sistemas de control de versiones (`git` los usa internamente) y de los parches. Comparar versiones de archivos de configuración es su uso diario.

Contraseña para **bandit18**: `0QxXZjELndr90zuh0TDYBEomI0SZITXI`

19. Nivel 18 – Sesión que se cierra sola

Objetivo: leer `readme` aunque el shell cierra la sesión inmediatamente después de iniciar.
Comando:

```
ssh -p 2220 bandit18@bandit.labs.overthewire.org "cat readme"
```

```
bandit18@bandit:~$ cat readme
KpsOfPkcP7i1F1IEExk2QEjyt6dw8dxZI
```

Explicación:

- Al conectar de forma interactiva, el servidor muestra un mensaje (*Byebye!*) y corta la sesión al instante: el shell configurado para esta cuenta sale apenas arranca.
- La clave está en que `ssh` acepta un **comando como argumento**: `ssh host "comando"` no abre un shell interactivo, sino que ejecuta ese comando directamente en el servidor y devuelve la salida.
- Como el comando corre antes de que el shell de login tenga oportunidad de cortar, `cat readme` funciona igual.
- Lección de automatización: pasar comandos a `ssh` sin sesión interactiva es la base de los scripts de administración remota.

Contraseña para bandit19: KpsOfPkcP7i1F1IEExk2QEjyt6dw8dxZI

20. Nivel 19 – Binario `setuid`

Objetivo: usar el binario `bandit20-do` para ejecutar comandos como el usuario `bandit20`.
Comandos:

```
ls -la
file bandit20-do
./bandit20-do cat /etc/bandit_pass/bandit20
```

```
bandit19@bandit:~$ ls -la && file bandit20-do && ./bandit20-do cat /etc/bandit_pass/bandit20
total 36
drwxr-xr-x  2 root   root   4096 Jun 24 14:58 .
drwxr-xr-x 150 root   root   4096 Jun 24 15:02 ..
-rw-r--r--  1 root   root    220 Feb 13 12:16 .bash_logout
-rw-r--r--  1 root   root   3051 Jun 24 14:50 .bashrc
-rw-r--r--  1 root   root    807 Feb 13 12:16 .profile
-rwxr-x---  1 bandit20 bandit19 14880 Jun 24 14:58 bandit20-do
bandit20-do: setuid ELF 32-bit LSB executable, Intel 386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, BuildID[sha1]-263f71465a75a599f568ae40f1caca2a9e937892, for GNU/Linux 3.2.0, not stripped
4p1jcuZ0FKZvmp3lwG8V77Vhxdop0A
```

Explicación:

- El listado revela la pista: `-rwsr-x---` 1 bandit20 bandit19 bandit20-do. El archivo es propiedad de `bandit20`, y en la posición del bit de ejecución del propietario aparece una `s` en lugar de `x`.
- Esa `s` es el bit **setuid**: cuando se ejecuta el programa, el sistema operativo le asigna el ID de usuario **del propietario** (`bandit20`) en lugar del de quien lo lanza (`bandit19`). Es el mecanismo clásico de Unix para escalar privilegios de forma controlada (así funcionan `sudo` y `passwd`).

- `file` confirma que es un ELF ejecutable (un binario real, no un script), con `setuid` activo.
- `./bandit20-do cat /etc/bandit_pass/bandit20` le pide al programa que ejecute `cat` sobre el archivo de contraseñas del nivel 20: como el binario corre como `bandit20`, puede leerlo y lo imprime.
- Lección de seguridad: `setuid` es poderoso y peligroso; un binario `setuid` mal diseñado es una puerta de escalada de privilegios. Este nivel muestra exactamente por qué.

Contraseña para bandit20: 4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6p0A

Resumen de contraseñas

Nivel	Contraseña para el siguiente nivel
0	6y2kwnwK6grgvwvpvLaa2T1cpFEK0hNR
1	PK8fYLZg2hnHSz83plBL1iEPKdD3QToB
2	7ZZ2LFrykP2zEyvB14m3clcL7tGYJPME
3	xzTXq1rDJQVVAzdV5cHq1TQytTWufAMq
4	6C7h9GD8M6ai5nr7wo1RonrzFjj9yIrG
5	pXa26xhMwac2SvDotA4r9EgZkul0eSBW
6	Bmnnvf82KzQlfxgAI2d1zYbr1u9pr3E3
7	VR1ljMayciFxbnUokuQmJFw6QC9VKtub
8	Ejm0SvuAu7sGAHqHVcBDPirRe9T03kx1
9	B0s2khmbT9u0geKu0oVGW3JZKhndE3BG
10	pYf0Y6HwUsDj5rL9UvyhU7MCmv8vN5Ro
11	GR0ozWP08QyN0mGrjUkID0WCYkZiQxrN
12	qQYQiHOBPR8zR61qxYqX45quvihF2uzk
13	aaWecNkG4FhxJQxz07uiwzVP6bJiYS65
14	pbLYuZtTg4MgaqfJx8jbA9gKKGqM68A7
15	kSOHf0u5HiXFwKMKFqXvPd0TNGGa0X8V
16	pWMAZoxGC8JmDMfmT5MGEsobMM3vnj2
17	0QxXZjELndr90zuh0TDYBEomIOSZITXI
18	Kps0fPkcP7i1F1IEk2QEjyt6dw8dxZI
19	4pIjcunZ0fK2vmp3IwfG8Vf7VhxD6p0A

Herramientas usadas (resumen rápido)

Comando	Para qué sirve
<code>cat</code>	Imprimir contenido de archivos
<code>ls -la</code>	Listar archivos, incluidos ocultos, con permisos
<code>file</code>	Identificar el tipo real de un archivo
<code>find</code>	Buscar archivos por nombre, tamaño, permisos, propietario
<code>grep</code>	Filtrar líneas por patrón
<code>sort / uniq</code>	Ordenar y detectar líneas únicas/duplicadas
<code>strings</code>	Extraer texto legible de binarios

Comando	Para qué sirve
<code>base64 -d</code>	Decodificar base64
<code>tr</code>	Traducir/rotar caracteres (ROT13)
<code>xxd -r</code>	Revertir un volcado hexadecimal
<code>gzip -dc / bzip2 -dc / tar xf</code>	Descomprimir y extraer archivos
<code>ssh -i</code>	Autenticarse con clave privada
<code>nc</code>	Conexiones TCP crudas
<code>openssl s_client</code>	Cliente TLS de prueba
<code>nmap</code>	Escanear puertos
<code>diff</code>	Comparar archivos
<code>2>/dev/null</code>	Silenciar errores (redirección de stderr)